

Survey on Differences of Requirements Engineering for Traditional and Agile Development Processes

Alhejab Alhazmi
The Department of Computer and Electrical Engineering and
Computer Science
Florida Atlantic University
Boca Raton, Florida, United States
aalhazmi2016@fau.edu

Dr. Shihong Huang
The Department of Computer and Electrical Engineering and
Computer Science
Florida Atlantic University
Boca Raton, Florida, United States
shihong@fau.edu

Abstract— Traditional and Agile are two different software development approaches, where recently the Agile have attracted the attention of the software development industry due to its capability to overcome the limitations of the traditional software development approaches and to cope with increasing complexity in system development. This paper presents a literature review that is focusing on requirements engineering in both traditional and Agile development processes with a discussion of their existing challenges. In addition, it illustrates how Agile approaches can help to overcome the challenges of managing requirements in traditional methods. The paper also shows the importance of activity planning of requirements engineering in improving the product quality, cost, and scheduling. Finally, the paper points out some drawbacks of Agile practices and available solutions.

Keywords— Traditional development processes, Agile development processes, Requirement Engineering (RE), Scrum, Extreme Programming

I. INTRODUCTION

Traditional and Agile are two different software development approaches. Traditional methodologies represent the software development methodologies that are based on pre-organized phases/stages of the software development lifecycle. Agile software development methods have attracted the attention of the software development industry and have become one of the most effective approaches due to its capability to overcome the limitations of the traditional software development approaches and to cope with increasing complexity in system development [1-8]. Agile software development methods promise benefits to include deliver products faster, with high quality and satisfy customer needs; thus, it aims to deliver business value in short iterations [9][10]. Therefore, in Agile software development methods, there is an advantage of possibility to change the direction of product development immediately based on the current situation, since the development process is carried out incrementally and empirically [8].

The requirements engineering (RE) process is the first activity in traditional methodologies where the system requirements are identified to illustrate what to build before system development starts. As we will discuss later, traditional methods face many challenges, such as communication gaps, and over-scoping, which are resolved by the Agile requirements engineering (RE) approach (Section IV).

A. Our Contributions

In this paper, we conduct a literary study for both traditional and Agile development processes with the focus on handling requirements engineering. Our contributions are as follows:

- A literature review that is focusing on handling requirements engineering in both traditional and Agile development processes.
- How Agile approaches help to overcome the challenges of managing requirements in traditional methods?
- Show the importance of activity planning of requirements engineering in improving the product quality, cost, and scheduling.
- The importance of addressing the drawbacks of Agile practices (e.g., minimum documentation) and available solutions.

B. Paper Organization

This paper is organized as follows: Section II presents a general background of the requirements engineering in both traditional and Agile development processes. Section III highlights the main related works. Section IV illustrates the challenges of the requirements engineering in both traditional and Agile development processes and the available solutions with explaining how Agile methods overcome the traditional's challenges. Section V discusses why the focus is on Agile, shows the importance of solving Agile challenges, and reviews our previous work which was focusing on planning optimization for Scrum project management. Finally, Section VI concludes the paper and points to future work.

II. BACKGROUND

A. Requirements Engineering in Traditional Methods

The term of traditional methodologies represents the software development methodologies that are based on pre-organized phases/stages of the software development lifecycle, such as waterfall model, spiral model, and V-model. Generally, requirements engineering is concerned with identifying, modeling, communicating and documenting the system requirements, and it defines the context of system usage. What to build before system development starts, is the main goal of requirements engineering, so that it is the process to ensure the requirements are complete, consistent and relevant. Developers are relying on many specialized techniques during the

requirements engineering process [11]. According to [12], the RE process has five activities: Elicitation, Analysis and Negotiation, Documentation, Validation, and Management.

1) *Requirements Elicitation*: Requirements elicitation activity is a process to discover requirements and identify boundaries of the system by defining its context based on consultation with stakeholders [3][11]. Therefore, in order to gain an understanding of the system to be developed and to get better implementation, it is essential to recognize the application domain, business needs, system constraints, and stakeholders, etc. The most important techniques for requirements elicitation are: Interviews, Use Cases / Scenarios, Observation and Social Analysis, Focus Groups, Brainstorming, and Prototyping [12].

2) *Requirements Analysis*: The activity of requirements analysis aims to check necessity, consistency, completeness, and feasibility of the requirements. Through the negotiation process with stakeholders, the requirements conflicts are resolved as well as the critical requirements are identified to be prioritized [3][11]. In this activity, there are three main techniques are adopted to analyze requirements, which are JAD, Prioritization, and Modeling [12].

- **Joint Application Development (JAD)**: It is a structured analysis approach during a workshop or a group session, where developers and customers discuss the required specifications of the product [11].
- **Requirements Prioritization**: This technique is to prioritize the most valuable features of the product's requirements which are essential to be delivered as early as possible, especially with a tight schedule, limited resources, and high customer expectations. The priority of the product's requirements has to be determined by the customer with the help of developers who may propose to implement lower priority specifications earlier due to their relationship with other high priority specifications [3][11].
- **Modeling**: Product's requirements are modeled to clarify the specifications for the developers, where it is adopted also to be a bridge between the analysis and the design process. Many modeling techniques can be used in the requirements analysis process, where the most famous techniques are: data-flow models, semantic data models and object-oriented approaches [3][11][12].

3) *Requirements Documentation*: The requirements documentation aims to communicate requirements between stakeholders and developers. It is used as the baseline for upcoming processes that are design, testing, verification, and validation activities as well as for change control [3][11]. Therefore, the document of requirements describes the product's specifications unambiguously, completely, correctly, understandably, consistently, concisely, and feasibly [11][12]. Thus, this document of requirements is often used as a be part of the contract.

4) *Requirements Validation*: The requirements validation means that the customer's needs are documented correctly and

completely; therefore, these requirements are certified as accepted specifications to be implemented [11][12]. Requirements testing and requirements reviews are techniques used for requirements validation.

5) *Requirements Management*: The requirements management aims to capture, store, disseminate, and manage information [11][12]. It includes all activities concerned with requirements change and version control, tracing, and status tracking [3]. In order to manage changes to a system, requirements traceability illustrates the relationships between requirements, design, and implementation of a system.

B. Requirements Engineering in Agile Methods

Recently, software development has moved from plan-driven to value-driven process models. In plan-driven environments, people negotiate about pricing models, project plans and the number of product features they can develop using available resources. In contrast, in value-driven environments, people discuss visions, experiences and human values as well as how they can address them through the product [8]. In Agile Manifesto, the priority should be given to “individuals and interaction over processes and tools”, “working software over comprehensive documentation”, “customer collaboration over contract negotiation”, and “responding to changes over following a plan” [7][13]. There are few Agile processes are adopted by industry, including Scrum, and XP.

In Agile methods, requirements engineering is informal and highly dependent on individuals' knowledge and skills [14]. Requirements engineering is performed iteratively during the development process instead of during a closed phase in the beginning [8]. Therefore, a just-in-time model is often used to define high-level requirements in low-level tasks that can be implemented by developers who work collaboratively with business people, stakeholders, and users. This model is based on artifacts and begins with capturing requirements as epics. An epic is a large user story [15], that can be refined by utilizing story maps [16]. A story map consists of user stories [17], which are split into tasks. Although there are many different Agile methods, Scrum and Extreme Programming (XP) are the most popular of them [10][15-20].

In Scrum, a product owner, who is a one-person, is responsible for requirements elicitation and prioritization [1][15-18]. In Scrum, a product backlog is the source of user requirements which are considered as items that may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed [1]. The product backlog is used as a container of new requirements to replace old requirements, fix bugs, or remove features. Elements at the top of the backlog are prioritized as more refined items than other items [21][22]. The product owner is relying on many factors that contribute to determining the priority within the Product Backlog, such as Priority Factor, Story Size Factor and Complexity Factor [1][23-26]. The product owner works with the development team to conduct requirements analysis, requirements specification and requirements validation informally and in collaboration. At the beginning of each development iteration (Sprint) which takes two to four weeks, items are selected by the product owner and the development

team from Product Backlog for the Sprint Backlog to be implemented [1][27]. At the end of the iteration (Sprint), the product owner will validate the system by reviewing the new system behavior [15][16][18].

In Extreme Programming (XP), the focus is on software construction itself instead of detailed requirements, where requirements engineering based on a little guidance to follow [19]. Planning game is the main requirements engineering practice where elicitation techniques like interviews, brainstorming, and prioritization are used to be recorded in the story cards [11]. XP follows the approach of small frequent releases with focusing on writing tests before coding. In the next release, the functions and design of the delivered program can be reviewed and tested by the customers, where it is a suitable time to discuss requirements they want to change or add [11].

III. RELATED WORK

We describe several studies with a similar focus related to us. These have been the basis for our investigation. In [5], the authors conducted a mapping study of requirements engineering in Agile software development to review the scientific literature. They identified 28 papers on the topic, and the results of their analyzing indicate that the definition of Agile requirements engineering is still vague. They focused on Agile requirements engineering benefits and challenges, and they also proposed solutions to some of the identified problems.

The authors, in [28], conducted a systematic review of the literature published between 2002 and 2013. They identified 21 articles that discuss Agile requirements engineering, and they identified Agile requirements engineering practices. Thereafter, they illustrated the challenges traceable to traditional requirements engineering that were overcome by Agile requirements engineering, and the challenges posed by the practice of Agile requirements engineering.

In [8], the authors presented a systematic literature review with an extensive quality assessment of the included studies. They identified 27 relevant articles, where they stated that Agile requirements engineering is a complex research field with cross-functional influences. Their study assessed the involvement of stakeholders and users in Agile requirements engineering, provided methodologies that make Agile software development more human-centric and giving an overview of requirements management in Agile software development.

In the article [11], authors compared traditional requirements engineering approaches with the requirements engineering process in Agile methods. The authors analyzed similarities and differences of both development processes approaches, and they discussed how requirements engineering methods can benefit Agile software development.

IV. CHALLENGES IN TRADITIONAL AND AGILE RE

A. The Challenges of Traditional RE that are Resolved by Agile RE

As it was reviewed in the background section, the traditional requirements engineering approach is considered as a set of sequential activities: Elicitation, Analysis and Negotiation, Documentation, Validation, and Management. This approach

has its own challenges that we will try to cover in this section, in addition to explain how these challenges are resolved by the Agile requirements engineering approach.

In general, the changes promoted by Agile methods have impacts on traditional software development stages where the focus here is on the requirements engineering activities. Therefore, there are many researchers have focused on understanding Agile methods impacts on the requirements engineering [28]. The dynamic nature of Agile allows stakeholders continuously changes, which means that the requirements arise constantly during the development process [29]. Authors, in [30], presented that in Agile methods changes are continuing, where phase-driven approaches of the emergence of requirements are not supported as traditional software development. Therefore, software practitioners and researchers are focusing on countering the challenges of traditional requirements engineering practices by relying on Agile methods. Through numerous literary reviews such as [28], the following are the main challenges are identified and discussed which are summarized in Table 1.

1) *Communication Gaps*: It can be summarized as an interruption in the provision of the required information to the concerned people [28]. In [31], authors attributed these gaps to unclear vision of goals and unclear roles. Therefore, in [32-34] the authors presented that the communication gaps of requirements engineering can be handled with Agile methods through its communication mechanism, which is frequent face-to-face communication, to overcome this challenge.

2) *Over-Scoping*: It is a definition of the project's scope that is too large for available resources [28]. In [31], the authors presented that over-scoping occurs because of many reasons, where the most important are scope changes after finalizing a project, communication gaps, and quality issues. To reduce the over-scoping in the project, authors in [32] suggested that the customer's interaction and intervention at each step of the process will enable the customer to give approval for the progress or to recommend changes; thus, this will guide developers to avoid unwanted and out-of-scope requirements.

3) *Requirements Validation*: According to [33], one of the biggest challenges of traditional requirements engineering is requirements validation. On the other hand, the continuous prioritization in Agile methods can handle the validation of requirements [35]. In addition, requirement changes are suggested and investigated at the end of each iteration, where the user story is discussed with the product owner and customer representatives, to guarantee that the user story meets what is required [28].

4) *Requirements Documentation*: It is one of the main challenges to traditional methods, whether in the reliability of documentation or even the accuracy of the content that is often modified during the development process [32][36]. In addition, it is difficult to compile detailed specification documents because they consume excessive amounts of time and thus may delay the overall development time. On the contrary, in Agile methods, documenting requirements rely on face-to-face communication and tacit knowledge rather than documents as

Table 1: The Challenges of Traditional Requirements Engineering that are Resolved by Agile Requirements Engineering

CHALLENGES	DESCRIPTION	HOW AGILE RE DEALS WITH THIS CHALLENGE
Communication Gaps	Interruption in the provision of the required information to the concerned people → unclear vision of goals and unclear roles	- Frequent face-to-face communication - Collocated teams - Onsite customer - Alternate customer representations - Cross-functional Agile teams - Integrated RE process
Over-Scoping	Determining the scope of the project that is too large for available resources → It occurs because of communication gaps, scope changes after finalizing a project, and quality issues	- One continuous scope flow - Gradual detailing
Requirements Validation	It is one of the biggest challenges of traditional requirements engineering	- The continuous prioritization - Requirement changes are suggested and investigated at the end of each iteration - Prototyping
Requirements Documentation	The reliability of documentation and the accuracy of content → Consuming excessive amounts of time	- Face-to-face communication and tacit knowledge - Documentation is used to include user stories, product backlogs, etc
Customer Involvement	Rare customer involvement "customers receive the product after development and testing" → Increase the cost of rework and reduce agreement between customers and the development team	- Customers remain aware of the progress during the development life cycle → They provide timely feedback for the development team - Requirements prioritization by the customer

in traditional ways to explicitly support the exchange of knowledge and decisions [29]. Therefore, Agile methods strongly support face-to-face communication to take the advantages of feedback and reviews with trying to shorten documentation, but that does not mean that Agile methods are completely discouraged the documentation, where they are used to include user stories, product backlogs, etc [28].

5) *Customer Involvement*: One of the main challenges of traditional methods is rare customer involvement, for instance in the waterfall model, customers receive the product after finishing development and testing [33]. In contrast, in Agile methods, customer involvement plays an important role, where customers remain aware of the progress during the development life cycle; thus, they provide timely feedback for the development team.[28]. As [37] explained, timely feedback from customers increases the agreement between customers and the development team as well as reduces the cost of rework.

B. The Practical Challenges of Agile RE and Proposed Solutions

Through our literature review that focuses on Agile requirements engineering approaches, although Agile requirements engineering practices help to overcome the challenges faced by traditional requirements engineering, it opens up many challenges for the software industry. In addition, Agile requirements engineering approaches create several constraints to achieve an adequate balance between agility and stability [32]. These challenges are discussed below with available solutions to overcome them, Table 2.

1) *Minimal Documentation*: It is a major challenge for development teams in Agile methods where the only documents in Agile methods are user stories and product backlogs [29][38][39]. In Agile methods, the traditional requirements documentation method is replaced with to-the-point, precise

and “user goal” oriented user stories [28][33]; therefore, in case of the absence of customer representatives, problems will arise when there are complexities in the project or unexpected changes in requirements due to lack of documentation. In some cases, such as working on large projects, verbal communication between the development team regarding requirements is insufficient, especially when the team’s work-space is not large enough to accommodate all members or they are in different places [40]. The problem of traceability is one of the most important effects of the lack (minimal) of documentation in Agile methods, as it is difficult for the development team to do tracing efficiently during the constant changes in requirements. Therefore, researchers and organizations are searching for a strategical solution to overcome this challenge. Currently, as a temporary solution, which remains not enough, user stories are defined with more detailed artifacts to help the development team make the right implementation options (called delivery stories) [40].

2) *Client or Customer Representatives Issues*: Agile requirements engineering relies heavily on interaction and trust between the customers and the developers, which may be easy to apply theoretically, but this assumption is often unrealistic [29][39][41]. This challenge has several aspects that are the customer availability, the customer competency, and the desire of the customer to spend resources on the ongoing verification of requirements and the solution system [41-43]. Empirical studies confirmed that availability and access to customers is a major challenge because of many factors from a business perspective such as cost, time, and workload of customer representatives [39][44][45]. Where the authors in [39]and [40], stated that customer inability refers to the incompetence of customer whether in decision-making or in domain knowledge. Consequently, the chances of disagreement between customer

Table 2: The Practical Challenges of Agile Requirements Engineering and Proposed Solutions

CHALLENGE	DESCRIPTION	IMPACT	PROPOSED SOLUTIONS
Minimal Documentation	- Most teams document only user stories and product backlogs	- The absence of customer representatives will arise problems - Difficulties for traceability	- User stories are defined with more detailed artifacts (delivery stories)
Customer Representatives	- The customer availability - The customer competency - The desire of the customer to spend resources on the ongoing verification of requirements and the solution system	- A disagreement between customer groups - Increase in rework	- Alternative or proxy customers to act as real customers (product owners in Scrum) while other organizations implement the "onsite developer" practice - Additional requirements documentation will be beneficial to overcome the overhead created by face-to-face communication - Mind-mapping for requirements elicitation
Imprecise Estimates	- Due to requirements changes and dynamic planning and design, prior accurate estimations for the time and the cost are impossible	- Project delays	- Frequent communication
Inappropriate Architecture	- The initial structure developed by the development team becomes inappropriate in later iterations with new requirements	- Changes occurring during later iterations - Complete rewrite of the system - Increase in cost	- There is no actual efficient solution
Non-Functional Requirements	- Ignoring non-functional requirements with relying on developers implementation on tacit knowledge	- Lower quality → Massive lapse and rework; thus, higher cost and longer time	- Accurate modeling approaches - The NORMATIC tool
Continuous Change of Requirements	- Agile methods allow requirements to change during development	- Increase in cost - Failure of project - Removing misleading and incorrect requirements based on changes requires additional cost and effort	Contracts: - Changes must be within agreed restrictions and do not conflict with the contracts concluded - Fixed payment per release protects investments and prevents a random change of requirements Changes: - Increase communication and customer engagement - RE-KOMBINE - JIRA
Insufficiency of the User Story Format	- It lacks the power of expression	- The properties such as consistency and verifiability difficult to be validated - Difficulties for traceability	- Delivery stories - Hierarchical requirements model - Aspect-oriented requirements approach - Feature-driven development based requirements analysis process - For traceability issue, traditional requirements analysis, specification process, and requirements traceability management are suggested
Difficulties in the Prioritization of Requirements	- Prioritization of requirements is based on the immediate business value	- Reaching consensus on prioritization of requirements can be difficult due to that customers may have conflicting needs	- Goal-based prioritization technique - Relaying on decision factors like Cost-Benefit to the Organization, Complexity, and Delivery Date/Schedule - Focusing on environment, process, and product - Depending on business value and process flow

groups will increase to impact the performance generally, especially in short development cycles [39]. Therefore, in practice, many Agile teams have alternative or proxy customers to act as real customers (product owners in Scrum), while other organizations implement the "onsite developer" practice by moving a developer representing to the customer site [28][40][45]. Moreover, [11] suggested that in large Agile development projects, additional requirements documentation will be beneficial to avoid the overhead founded due to face-to-face communication between the client or customer representatives and the development teams. In Scrum, mind-mapping is a suggested solution to be adopted in [46], which is a solution for requirements elicitation, where the initial results

stated that customers who receive training in requirements elicitation using a mind-map produce more requirements comparing with the customers who don't receive the same training.

3) *Imprecise Estimates*: In an Agile methods environment, budget and schedule estimations are important challenges that are not completely solved, which are often required by management [28][29][39]. Due to requirements changes and dynamic planning and design, prior accurate estimations are not possible, although the practice of Agile methods enables preliminary project evaluations. In some cases, the estimations are significantly affected when some of the user stories of a project may be discarded in forthcoming iterations [29][39].

4) *Inappropriate Architecture*: As Agile methods rely on continuous iterations, the initial structure developed by the development team becomes inappropriate in later iterations with new requirements [29][39][42]. In Agile environments, refactoring is an ongoing activity, but this refactoring may be extended to the entire system architecture; thus rewrite the complete system [39]. However, if refactoring were avoided and stopped, changes occurring during later iterations would add to the cost and become cumbersome to deal with [28]. In the articles, there are no solutions were proposed to solve this issue.

5) *Non-Functional Requirements*: Ignoring non-functional requirements with relying on their implementation on tacit knowledge [29][39], is a major challenge to Agile methods which means neglecting system quality. System quality includes internal quality and external quality. The internal quality of the systems refers to maintainability and testability ... etc, where the external quality of the systems refers to usability and security ...etc. Thus, failure to include system quality leads to a massive lapse and rework, which means higher cost and longer time [39][47]. In [48], authors proposed novel artifacts for Agile processes to include non-functional requirements through using accurate modeling approaches of user stories such as Agile use cases, Agile loose cases, and Agile choose cases. NORMATIC is a proposed tool in [49], where it depends on the modeling non-functional requirements for Agile methods.

6) *Continuous Change of Requirements*: Although Agile methods allow requirements change during development, the changes can be the reason for increasing the cost or even failure of projects; therefore, these changes must be within agreed restrictions and does not conflict with the contracts concluded. Thus, to avoid these possible effects and appropriately deal with the flexible nature of Agile requirements engineering, legal measures should be taken to protect all parties and clarify the mechanisms used. Therefore, organizations and development teams generally overcome these problems by fixed payment per release, which protects investments and prevents a random change of requirements [40]. Similarly, removing misleading and incorrect requirements based on changes requires additional cost and effort, where this attitude can be managed through constant communication and customer engagement to prevent subsequent reworking [28]. Recently, RE-KOMBINE framework has been proposed to handle requirements specification, where it allows the requirements to be formally specified, yet flexible enough to accommodate changes [50], in addition to the specific tools to deal with Agile requirements engineering such as JIRA [51].

7) *Insufficiency of the User Story Format*: A user story is the smallest unit of work in an Agile framework which is an end goal, not a feature. With user stories the properties such as consistency and verifiability difficult to validate [29][39]. User stories generally lack the power of expression, sometimes user stories do not convey enough information to design software, especially when the software system is large and complex or

hardware-dependent [40][52]. In addition, regular user stories do not explicitly support requirements traceability, which causes traceability issues when requirements, code and, tests are changed over time [42][52][53]. In order to overcome these issues there are some proposed ideas: The extension of user stories is one solution proposed by [40] to improve the developers' teams understanding of the requirements through delivery stories which contains functional specifications, high-level design, and test scenarios. Thus, the insufficiency of the user story format will be mitigated generally. Another solution was presented in [54], where the authors suggested the using of hierarchical requirements model, explicit product and portfolio management processes to improve the understanding of the requirements with sufficient details for developers. In addition, the aspect-oriented requirements approach in [55], and feature-driven development based requirements analysis process in [56], are proposed to overcome the insufficiency of the user story format in different aspects. For traceability, traditional requirements analysis, specification process, and requirements traceability management are suggested to be adopted in [52][57], especially with large and complex or hardware-dependent systems.

8) *Difficulties in the Prioritization of Requirements*: Prioritization of requirements relays on the immediate business value. Therefore, reaching a consensus on prioritization of requirements can be difficult due to that customers may have conflicting needs[29][32][39][42][58]. In Agile development, the customers' capability of directly prioritizing requirements are different and maybe limited [41]. In [59], the authors stated that since there is no detailed requirement analysis, the options of quantitative real approach for prioritizing requirements may not be possible in in Agile methods. Recently, researchers have focused on this issue and come up with many suggested approaches. [60] introduced their goal-based prioritization technique which is based on collecting relative weight of requirements with respect to goals identified from the stakeholders. [61] suggested decision factors like Cost-Benefit to the Organization, Complexity, Requirement Dependencies, and Delivery Date/Schedule as the most important factors for prioritizing the user stories. Authors in [62] highlight the three major aspects environment, process, and product, where each one consisting of further factors in order to prioritize requirements. In [63] authors provided an approach to tackle the issue of prioritization considering both the business value and process flow aspects and help the product owner in making decisions about which requirements to be considered first and arranging them in the product backlog.

V. DISCUSSION

As was mentioned earlier, Agile software development methods have attracted the attention of the software development industry and have become one of the most effective approaches due to its capability to overcome the limitations of the traditional software development approaches and to cope with increasing complexity in system development [1-8]. There are many articles describing the benefits of Agile

requirements engineering approaches comparing with the traditional requirements engineering processes [3]. These benefits can be summarized as follows, Table 3:

1- *Lower process overheads*: Process overheads are lowered because it lacks time-consuming processes, especially lengthy documentation; thus, reducing the need to reformulate requirements. [5][11][29][32][64].

2- *Improved requirements understanding*: Immediate access for customers provides better understanding of requirements and their priorities as well as a faster validation due to the direct communication with customers which is no longer an obstacle today, both in terms of distributed or global software development contexts [5][29][32][39][42][43][65].

3- *Reduced resources overallocation*: Agile requirements engineering approaches tend to reduce the over-allocate development resources by enabling better management of the development scope [5][32][42][65].

4- *Responsiveness to change*: Agile requirements engineering approaches are generally flexible, with highly responsive to changes in both content and requirements' priority. Where changes are highly likely due to rapid change in the software environment or customer understanding [5][11][29][39][42][64].

5- *Rapid delivery and validation*: Agile requirements engineering approaches allow rapid delivery of value after validating the system during the development iterations [5][11][29][64].

6- *Improved customer relationships*: The relationship between customers and users with developer teams is improved in Agile requirements engineering approaches due to their continuous and direct communication during the software development process. [5][29][42].

Table 3: Agile Benefits

BENEFITS	DESCRIPTION
Lower Process Overheads	It lacks time-consuming processes, especially lengthy documentation → reducing the need to reformulate requirements
Improve Requirements Understanding	Immediate access for customers provides: - A better understanding of requirements and their priorities - Faster validation due to the direct communication with customers
Reduce Resources Overallocation	- Reducing the over-allocate development resources through enabling better management of the development scope
Responsiveness to Change	- Agile methods are flexible, with highly responsive to changes in both content and requirements' priority
Rapid Delivery and Validation	- Agile RE approaches allow rapid delivery of value after validating the system during the development iterations
Improve Customer Relationship	- Due to continuous and direct communication between customers and users with developer teams during the software development process

A. The Importance of Solving Agile Challenges

Previously, the discussion illustrated that traditional methodologies have many challenges, especially those are

related to requirements engineering; however, Agile methods can overcome these challenges by depending on their characteristics. Through these important challenges faced by traditional methodologies and its impacts on the product from different aspects, the importance of resolving them is clear. The Agile methods provide effective solutions to deal with these challenges as a part of its framework. However, the adoption of Agile methods creates many of its own challenges.

As shown above, Agile methods challenges have appeared with the increasing of its prevalence among developers; therefore, working on resolving these issues is very important which will in turn reflect on Agile reliance and trust in a software development environment. Currently, there are proposed solutions to overcome these issues, but they still need more practical applying in different situations, such as large, distributed, and outsourced projects, with tracking their effects whether they are efficient or not. In general, more effort needs to be put into evaluating case studies, to better assess the usability of Agile methods with focusing on the diversity of available solutions for the defined issues. Thus, working to find more customized tools for these issues will certainly have a positive impact on Agile methods.

B. Our Work

In our work, we generally focus on planning optimization for project management, hence the selection of work on the Scrum framework, which is considered as a project management approach. In previous work [1], we proposed the use of Sprint Planning dEcision Support System (SPeSS) which is a tool to assist the managers for Sprint planning in the Scrum framework. Among considering other Sprint planning factors, SPeSS takes into consideration developer competency, developer seniority and task dependency, Figure 1. The results are that the assignments of the tasks of each Sprint to developers guarantee that each team member contributes to their fullest potential, and project planning is optimized for the shortest possible time.

In the Scrum framework, we can notice that there is a possibility to improve the efficiency of the framework through adding tools to be adopted as well as mechanisms to be followed in order to overcome Scrum's defects primarily. In addition to rise quality, improve scheduling, and reduce the cost that are

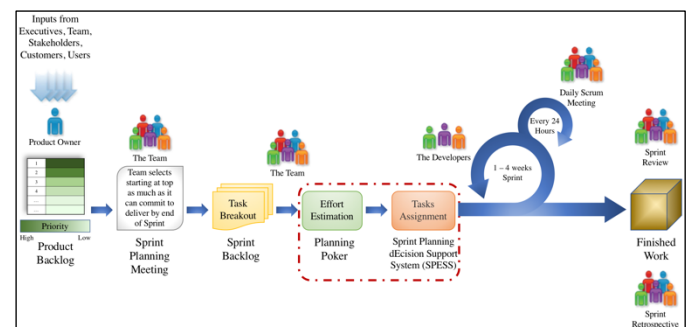


Figure 1: The Improved Scrum: It is a regular Scrum framework with adding new components that are Planning Poker and Sprint Planning dEcision Support System (SPeSS) tool. The Sprint Planning dEcision Support System (SPeSS) is a tool to assist the managers for Sprint planning that combines the developer competency, the developer seniority and the task dependency with using the estimated efforts that comes from planning poker for each task. This tool assigns the tasks of each Sprint to developers in the Sprint, so the developers will work on their assigned tasks.

related to Scrum with the possibility to affect other Agile methods positively. In this context, this paper focuses on the early stages of the software development process, especially Requirements Engineering. Therefore, we have conducted a concentrate literary review in this area, which reviewed the most important issues faced by traditional methodologies in the requirements engineering stage as well as how the Agile methods deal with these issues. Thereafter, we discussed the issues faced by Agile methods with its proposed solutions from other researchers. Thus, we will build on this work to take other steps on planning optimization for Scrum project management.

VI. CONCLUSION

In this paper, we conducted a literature review that is focusing on requirements engineering for Agile and traditional methods. The most important benefits of adopting Agile methods are listed which include, lowering process overheads, improving requirements understanding, reducing resources overallocation, highly responsive to change, rapid delivery and validation, and improving customer relationships. Thereafter, we reviewed how Agile methods were able to overcome the requirements engineering challenges mentioned in traditional methodologies. Where Agile methods are relying on frequent face-to-face communication with the customer, the customer's intervention and interaction at each step of the process, the continuous prioritization in Agile methods which may be changed based on the business value or other factors, requirement changes are suggested and investigated at the end of each iteration to be implemented in the next iterations, documenting requirements rely on face-to-face communication and tacit knowledge with constant feedback and reviews, and customers remain aware of the progress during the development life cycle; thus, they provide timely feedback for the development team.

Likewise, Agile methods have its challenges, where the most important challenges and the proposed solutions so far, are discussed, in addition to recognize the researchers' and organizations' interests, which are to overcome these challenges and balance Agile methods. Therefore, in our work, we generally focus on planning optimization for project management, hence the selection of work on the Scrum framework, which is considered as a project management approach. In previous work [5], we proposed the use of Sprint Planning dEcision Support System (SPESS) which is a tool to assist the managers for Sprint planning in the Scrum framework. In the Scrum framework, we can notice that there is a possibility to improve the efficiency of the framework through adding tools to be adopted as well as mechanisms to be followed in order to overcome Scrum's defects primarily. In addition to increase efficiency, improve time, and rise quality that are related to Scrum with the possibility to affect Agile methods positively. In this context, this paper focuses on the early stages of the software development process, especially Requirements Engineering.

In future work, we will build on this work to take other steps on planning optimization for Scrum project management. Where the focus will be on items prioritization on Product Backlog with taking into account the dependency among the items. In addition, we will work to find a suitable mechanism for

requirements traceability, which is one of the most important defects of Agile methods generally due to frequent changes in requirements with lacking detailed documentation.

REFERENCES

- [1] A. Alhazmi and S. Huang, "A Decision Support System for Sprint Planning in Scrum Practice," SoutheastCon 2018, St. Petersburg, FL, 2018, pp. 1-9.
- [2] [1] S. Sharma and N. Hasteer, "A Comprehensive Study on State of Scrum Development," Proceeding - IEEE Int. Conf. Comput. Commun. Autom. ICCCA 2016, pp. 867-872, 2016.
- [3] A. Batool et al., "Comparative study of traditional requirement engineering and Agile requirement engineering," 2013 15th International Conference on Advanced Communications Technology (ICACT), PyeongChang, 2013, pp. 1006-1014.
- [4] P. Abrahamsson, O. Salo, J. Ronkainen, and J. Warsta, "Agile software development methods: Review and analysis," VTT Publications. 2002.
- [5] V. T. Heikkilä, D. Damian, C. Lassenius and M. Paasivaara, "A Mapping Study on Requirements Engineering in Agile Software Development," 2015 41st Euromicro Conference on Software Engineering and Advanced Applications, Funchal, 2015, pp. 199-207.
- [6] T. Poppendieck, M. Poppendieck, "Lean Software Development: An Agile Toolkit for Software Development Managers", 2003 Addison-Wesley.
- [7] A. Aurum and C. Wohlin, Engineering and managing software requirements. 2005.
- [8] E. M. Schön, J. Thomaschewski, and M. J. Escalona, "Agile Requirements Engineering: A systematic literature review," Comput. Stand. Interfaces, 2017.
- [9] G. Kotonya and I. Sommerville, "Requirements Engineering : Processes and Techniques (Worldwide Series in Computer Science)," Star, 1998.
- [10] T. Dybå and T. Dingsøyr, "Empirical studies of Agile software development: A systematic review," Information and Software Technology. 2008, pp. 833-859.
- [11] [5] F. Paetsch, A. Eberlein, and F. Maurer. Requirements engineering and Agile software development. In Proc. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises (WET ICE), 2003, pp. 308-313.
- [12] G. Kotonya and I. Sommerville, "Requirements engineering with viewpoints," Software Engineering Journal. 1996, pp. 5-18.
- [13] K. Beck et al., "Manifesto for Agile Software Development," The Agile Alliance, 2001.
- [14] T. Dingsøyr, S. Nerur, V. Balijepally, and N. B. Moe, "A decade of Agile methodologies: Towards explaining Agile software development," Journal of Systems and Software. 2012, pp. 1213-1221.
- [15] M. Cohn, User Stories Applied: For Agile Software Development (Addison Wesley Signature Series). 2004.
- [16] J. Patton, User Story Mapping. Discover the Whole Story, Build the Right Product. 2014.
- [17] W. Hudson, "User stories don't help users," interactions, 2013, pp. 50-53.
- [18] P. Deemer, G. Benefield, C. Larman, and B. Vodde, "The Scrum Primer: A Lightweight Guide to the Theory and Practice of Scrum," version 2. Technical report, 2012.
- [19] Kent Beck; Cynthia Andres, "Extreme Programming Explained: Embrace Change, Second Edition," XP Ser., 2004.
- [20] S. Stavru, "A critical examination of recent industrial surveys on Agile method usage," Journal of Systems and Software, 2014.
- [21] A. M. Alsalemi and E. T. Yeoh, "A survey on product backlog change management and requirement traceability in Agile (Scrum)," 9th Malaysian Software Engineering Conference, MySEC 2015.
- [22] C. Ashbacher, "Succeeding With Agile: Software Development Using Scrum, by Mike Cohn,," Journal of Object Technology., 2010.
- [23] R. Popli, N. Chauhan, and H. Sharma, "Prioritising User Stories in Agile Environment," in Proceedings of the 2014 International Conference on Issues and Challenges in Intelligent Computing Techniques, ICICT 2014, 2014, pp. 515-519.

- [24] K. Logue, K. McDaid, and D. Greer, "Allowing for Task Uncertainties and Dependencies in Agile Release Planning," 4th Proc. Softw. Meas. Eur. Forum, 2007, pp. 275–284.
- [25] S. Downey and J. Sutherland, "Scrum Metrics for Hyperproductive Teams: How They Fly Like Fighter Aircraft," in Proceedings of the Annual Hawaii International Conference on System Sciences, 2013, pp. 4870–4878.
- [26] A.R.Mukker,L.Singh,andA.K.Mishra,"SystematicReviewofMetrics in Software Agile Projects," COMPUSOFT, An Int. J. Adv. Comput. Technol., vol. 3, no. 2, 2014, pp. 533–539.
- [27] J. Sutherland and K. Schwaber, "Scrum Guides." [Online]. Available: <http://www.scrumguides.org>.
- [28] I. Inayat, S. S. Salim, S. Marczak, M. Daneva, and S. Shamshirband, "A systematic literature review on Agile requirements engineering practices and challenges," Computers in Human Behavior. 2015.
- [29] L. Cao and B. Ramesh. Agile requirements engineering practices: An empirical study. Software, 2008, pp. 60–67.
- [30] E. Wolfgang, "Working with user stories". In Agile requirements engineering workshop, July 2011.
- [31] E. Bjarnason, K. Wnuk and B. Regnell, "Requirements are slipping through the gaps — A case study on causes & effects of communication gaps in large-scale software development," 2011 IEEE 19th International Requirements Engineering Conference, Trento, 2011, pp. 37–46.
- [32] E. Bjarnason, K. Wnuk, and B. Regnell, "A case study on benefits and side-effects of Agile practices in large-scale requirements engineering,". 1st Workshop on Agile Requirements Engineering (AREW '11), New York, NY, USA, 2011, ACM, pp. 3:1–3:5.
- [33] D. Carlson and P. Matuzic, "Practical Agile requirements engineering," in 13th Annual systems engineering conference, 2010.
- [34] A. Sillitti, M. Ceschi, B. Russo and G. Succi, "Managing uncertainty in requirements: a survey in documentation-driven and Agile companies," 11th IEEE International Software Metrics Symposium (METRICS'05), Como, 2005, pp. 10–17.
- [35] E. Hasnain and T. Hall, "Introduction to stand-up meetings in Agile methods," in AIP Conference Proceedings, 2009.
- [36] L. Jun, W. Qiuzhen, and G. Lin, "Application of Agile requirement engineering in modest-sized information systems development," in Proceedings - 2010 2nd WRI World Congress on Software Engineering, WCSE 2010.
- [37] M. Huo, J. Verner, L. Zhu, and M. A. Babar, "Software quality and Agile methods," in Proceedings - International Computer Software and Applications Conference, 2004.
- [38] [26] Y. Zhu, "Requirements Engineering in an Agile Environment," in Second world congress on software engineering, 2010.
- [39] B. Ramesh, L. Cao, and R. Baskerville. Agile requirements engineering practices and challenges: an empirical study. Information Systems Journal, 2010, pp. 449–480.
- [40] M. Daneva, E. van der Veen, C. Amrit, S. Ghaisas, K. Sikkil, R. Kumar, N. Ajmeri, U. Ramteerthkar, and R. Wieringa. Agile requirements prioritization in large-scale outsourced system projects: An empirical study. Journal of Systems and Software, 2013, pp. 1333–1353.
- [41] Z. Racheva, M. Daneva, K. Sikkil, A. Herrmann, and R. Wieringa. Do we know enough about requirements prioritization in Agile projects: Insights from a case study. In Proc. 18th IEEE International Requirements Engineering Conference, 2010, pp.147–156.
- [42] N. A. Ernst and G. C. Murphy. Case studies in just-in- time requirements analysis. In Proc. Second International Workshop on Empirical Requirements Engineering (EmpiRE), IEEE, 2012, pp. 25–32.
- [43] [10] R. Noordeloos, C. Manteli, and H. van Vliet, "From RUP to Scrum in global software development: A case study," Seventh International Conference on Global Software Engineering (ICGSE), IEEE, 2012, pp. 31–40.
- [44] M. Pichler, H. Rumetshofer and W. Wahler, "Agile Requirements Engineering for a Social Insurance for Occupational Risks Organization: A Case Study," 14th IEEE International Requirements Engineering Conference (RE'06), Minneapolis/St. Paul, MN, 2006, pp. 251–256.
- [45] Z. Racheva, M. Daneva, and A. Herrmann, "A conceptual model of client-driven Agile requirements prioritization: Results of a case study," in ESEM 2010 - Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement, 2010.
- [46] I. Mahmud and V. Veneziano, "Mind-mapping: An effective technique to facilitate requirements engineering in Agile software development," 14th International Conference on Computer and Information Technology, 2011, pages 157–162.
- [47] Cardinal, M. (2011). Addressing Non-Functional Requirements with Agile Practices. [online] Agilealliance.org. Available at: <https://www.Agilealliance.org/wp-content/uploads/2016/01/Addressing-NFR-with-Agile-practices-English-dec-16th.pdf> [Accessed 27 Nov. 2019].
- [48] W. M. Farid and F. J. Mitropoulos, "Novel lightweight engineering artifacts for modeling non-functional requirements in Agile processes," in Conference Proceedings - IEEE SOUTHEASTCON, 2012.
- [49] [33] W. M. Farid and F. J. Mitropoulos, "NORMATIC: A visual tool for modeling non-functional requirements in Agile processes," in Conference Proceedings - IEEE SOUTHEASTCON, 2012.
- [50] N. A. Ernst, A. Borgida, I. J. Jureta, and J. Mylopoulos, "Agile requirements engineering via paraconsistent reasoning," Inf. Syst., 2014.
- [51] JIRA, "JIRA issue and project," Atlassian Pvt Ltd., 2013.
- [52] J. Savolainen, J. Kuusela, and A. Vilavaara, "Transition to Agile development — rediscovery of important requirements engineering practices,". 18th IEEE International Requirements Engineering Conference, 2010, pp. 289– 294.
- [53] A. Lucia and A. Qusef, "Requirements engineering in Agile software development," Journal of Emerging Technologies in Web Intelligence, 2010.
- [54] K. Vlaanderen, S. Jansen, S. Brinkkemper, and E. Jaspers, "The Agile requirements refinery: Applying SCRUM principles to software product management," Information and Software Technology, 2011, pp. 58–70.
- [55] J. Araújo and J. C. Ribeiro, "Towards an aspect-oriented Agile requirements approach," Eighth International Workshop on Principles of Software Evolution, 2005, pp. 140– 143.
- [56] X. Ge, R. F. Paige, F. A. C. Polack, H. Chivers, and P. J. Brooke, "Agile development of secure web applications," 6th International Conference on Web Engineering (ICWE '06), New York, NY, USA, ACM, 2006, pp. 305–312.
- [57] D. Firesmith, "Generating complete, unambiguous, and verifiable requirements from stories, scenarios, and use cases," Journal of Object Technology, 2004, pp. 27–39.
- [58] O. Ktata and G. Le Vesque, "Agile development: Issues and avenues requiring a substantial enhancement of the business perspective in large projects," 2nd Canadian Conference on Computer Science and Software Engineering (C3S2E '09), New York, NY, USA, ACM, 2009, pp. 59–66.
- [59] Z. Racheva and M. Daneva, "How do real options concepts fit in Agile requirements engineering?," Eighth ACIS International Conference on Software Engineering Research, Management and Applications (SERA), 2010, pp. 231–238.
- [60] M. A. A. Elsood, H. A. Hefny, and E. S. Nasr, "A goal-based technique for requirements prioritization," in 2014 9th International Conference on Informatics and Systems, INFOS 2014.
- [61] G. Hoff, A. Fruhling, and K. Ward, "Requirement Prioritization Decision Factors for Agile Development Environments", in conference at Rawalpindi, Pakistan, IEEE, 2017 pp. 50 – 55.
- [62] R. H. AL-Ta'ani and R. Razali, "Prioritizing Requirements in Agile Development: A Conceptual Framework," Procedia Technol., 2013.
- [63] S. Sachdeva, A. Arya, P. Paygude, S. Chaudhary, and S. Idete, "Prioritizing User Requirements for Agile Software Development," in 2018 International Conference On Advances in Communication and Computing Technology, ICACCT 2018.
- [64] I. Sommerville, "Integrated requirements engineering: a tutorial. Software," IEEE, 2005, pp. 16–23.
- [65] E. Bjarnason, K. Wnuk, and B. Regnell, "Are you biting off more than you can chew? a case study on causes and effects of overscoping in large-scale software engineering," Information and Software Technology, 2012, pp.1107–1124, 10.